

Similarity Search

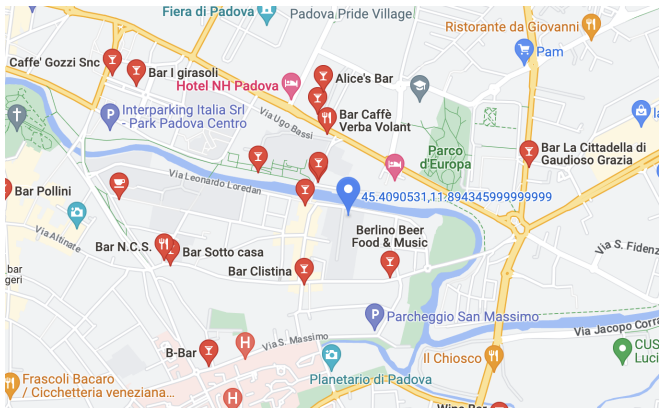
(Part 1)

OUTLINE

- ① Introduction to similarity search
- ② Similarity search in low dimensions
 - kd-trees
 - Curse of dimensionality
- ③ Similarity search in high dimensions (Part 2)
 - Approximate Near Neighbor (ANN) search
 - Locality Sensitive Hashing (LSH)

Introduction to similarity search

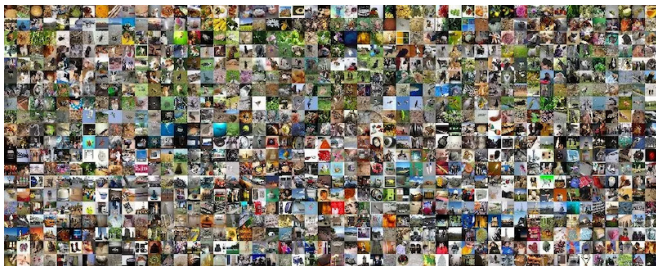
Searching near POIs



DATA: Points of interest (POIs) in a map, a distance function, a threshold r

GOAL: Find POIs near my current position (e.g., distance $\leq r$)

Searching similar images



DATA: A set of images, a distance function, a threshold r

GOAL: Find images similar to a given image (i.e., within distance $\leq r$)

- Similarity can be computed by embedding images into high-dimensional vectors with a machine learning model, and then compute the distance between the respective vectors (e.g., Euclidean distance, angular distance, ...)
- Similar problem for songs (e.g., Shazam)

Similarity Search

- The two case studies above are **two examples of similarity search**: given a set of objects, find items that are similar (or near).
- **A distance function is required** to evaluate the similarity of two objects: two objects are similar (or near) if their distance is below a given threshold.
- Main difference between the two examples: points in the map have **low dimensions** (points belong to $\mathbb{R}^2$), images are represented by vectors with **high dimensions** (e.g. in $\mathbb{R}^c$ with $c \gg 100$).

Similarity Search: applications

Some applications:

- **Information retrieval:** search similar documents, images, ...
- **Recommendation systems:** provide suggestions based on behaviors of similar users.
- **Entity resolution:** match equal or similar records.
- **Plagiarism detection:** match documents (e.g., papers, exams) with similar structure.
- **Outlier detection:** assign to each element and outlier score based on the distance from its k -th nearest neighbor.

r -Near Neighbor Search

For a given metric space (M, d) , point $q \in M$, and distance threshold $r > 0$ define the ball of radius r around q as

$$B_r(q) = \{p \in M : d(p, q) \leq r\}$$

Definition: r -Near Neighbor Search (r -NNS) problem

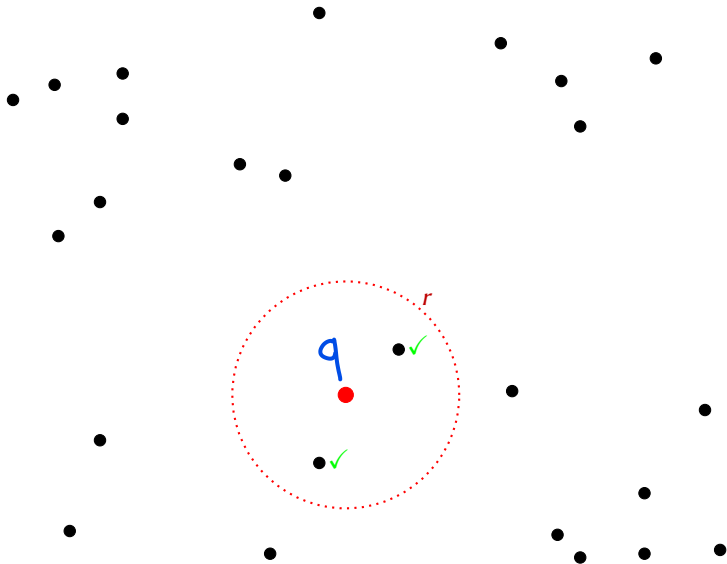
Given a set P of n points from the metric space (M, d) , construct a data structure that, given a query point $q \in M$ and a distance threshold $r > 0$, returns:

- a point $p \in B_r(q) \cap P$, if any such point exists;
- null if $B_r(q) \cap P = \emptyset$.

Observations:

- The query is unknown at the construction of the data structure
- If there are more points in $B_r(q) \cap P$, an arbitrary one is returned.
- The distance function and the threshold r depend on the application.

Example of r -NNS ($M = \mathbb{R}^2$, $d = || \cdot ||$)



Other similarity search problems

There are several other variants of similarity search problems e.g.,:

- **r -near neighbor reporting**: given the set P , a query point q , and a threshold r , return *all* points in $B_r(q) \cap P$.
- **Nearest neighbor search**: given the set P and query point q , return the *closest point* to q in P .
- **k -nearest neighbor search**: given the set P , a query point q , and an integer $k \geq 1$, return the k closest points to q in P .
- **Similarity join**: given two sets P and Q and a threshold r , return pairs $p \in P, q \in Q$ with $d(p, q) \leq r$.

Remark: searching for the nearest neighbor (or k -nearest neighbors) does not require a threshold r ; however, it is usually performed by reducing it instances of r -NNS, using a suitable sequence of values of r .

Computational setting

To introduce the main techniques, we will consider the **sequential setting** using the following **performance indicators**:

- **Construction time**: the time to construct the initial data structure;
- **Space**: the space (in memory words) to store the data structure;
- **Query time**: the time to execute one query.

For points in $\mathbb{R}^D$ we assume that $d(x, y)$ can be computed in $O(D)$ time and that a point is stored in $O(D)$ space.

However, **for very large inputs distributed or streaming solutions are often necessary**. This is an active research area.

Brute force solution

Construction

- Store points of P in a list.

Query q

- Compute $d(q, p)$ for all $p \in P$ returning the first point $p' \in P$ with $d(q, p') \leq r$, if one exists, and returning null, otherwise.

If $P \subset \mathbb{R}^D$ and it contains n points, the brute force approach requires:

- $O(n)$ construction time;
- $O(Dn)$ space;
- $O(Dn)$ query time.

IMPORTANT: The query time is too high!

Similarity search in low dimensions

r -NNS and Range Reporting in $\mathbb{R}^D$

To solve more efficiently the r -NNS problem in $\mathbb{R}^D$ under Euclidean distance, we use the **kd-tree**

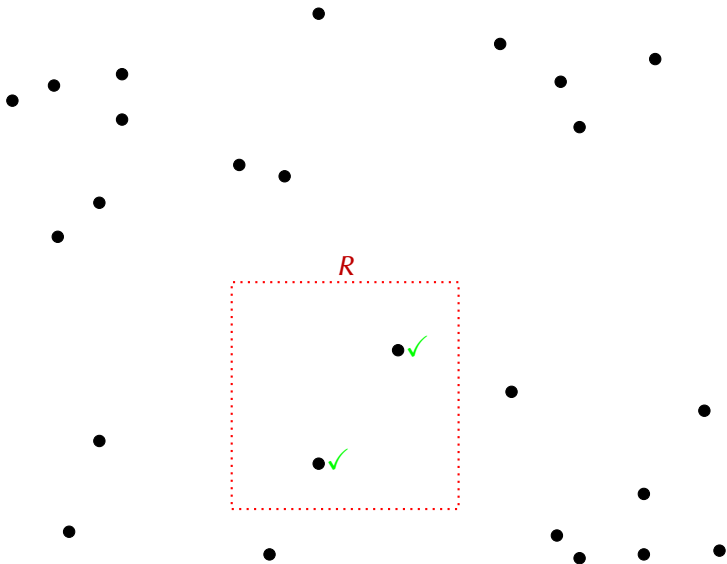
- A **kd-tree** is a **binary tree** that recursively partition the space into rectangular regions.
- kd-tree solves a slightly different problem named **Range Reporting**.

Definition: Range Reporting (RR) problem

Given a set $P \subset \mathbb{R}^D$ of n points, construct a data structure that, given a rectangular region R , returns **all** points of P contained in R .

Note: a rectangular region $R = [x_{1,1} : x_{1,2}] \times \dots \times [x_{D,1} : x_{D,2}] \subset \mathbb{R}^D$ is the region including all points $p = (p_1, \dots, p_D)$ with $p_i \in [x_{i,1} : x_{i,2}]$ for all $i \in \{1, \dots, D\}$.

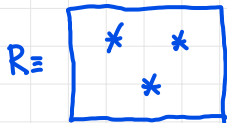
Range Reporting in $\mathbb{R}^2$



Range Reporting vs r -NNS

RR

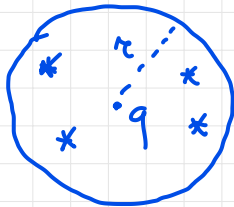
Query $\equiv$ Rectangle R



Task: return all points
in $R \cap P$

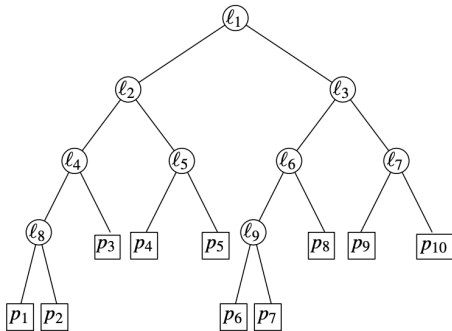
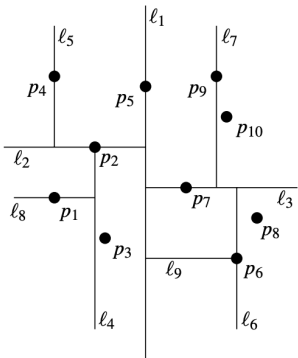
r -NNS

Query $\equiv$ point q , distance r



Task: return an arbitrary
point in $B_r(q) \cap P$,
if any

kd-tree



kd-tree: definition

For simplicity we define the kd-tree for the case $D = 2$. The definition can be extended to $D > 2$.

Let P be a set of n points in $\mathbb{R}^2$, and let $R(P) \subset \mathbb{R}^2$ be a rectangular region containing all points in P

A kd-tree T for P is a binary tree which defines a hierarchical decomposition of $R(P)$ into nested rectangular subregions. Namely,

- Each internal node v is associated to a rectangular region denoted as $\text{region}(v)$, and represents the subset of points $P_v = P \cap \text{region}(v)$.
- For the root r , we have that $\text{region}(r) = R(P)$, and $P_r = P$.
- Each external node (leaf) v is associated to a region containing a single point, which we denote as $p_v \in P$: i.e., $P_v = \{p_v\}$.

The regions associated to the nodes are defined by the following **splitting rule**.

Each internal node v with $|P_v| \geq 2$ has two children, u and w , whose respective regions, $\text{region}(u)$ and $\text{region}(w)$, are obtained by splitting $\text{region}(v)$ through a **vertical or horizontal line** ℓ_v , depending on v 's depth.

- If v is at **even depth**, ℓ_v is **vertical** at abscissa x_v . In this case:

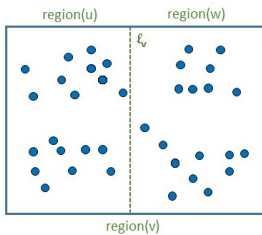
$$\begin{aligned}\text{region}(u) &= \text{region}(v) \cap \mathbb{R}_{x \leq x_v}^2 \\ \text{region}(w) &= \text{region}(v) \cap \mathbb{R}_{x > x_v}^2\end{aligned}$$

- If v is at **odd depth**, ℓ_v is **horizontal** at ordinate y_v . In this case:

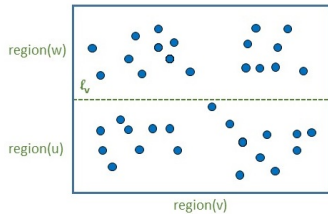
$$\begin{aligned}\text{region}(u) &= \text{region}(v) \cap \mathbb{R}_{y \leq y_v}^2 \\ \text{region}(w) &= \text{region}(v) \cap \mathbb{R}_{y > y_v}^2\end{aligned}$$

The line ℓ_v is chosen so to split the points in P_v into two sets (i.e., P_u and P_w) of size $\lceil |P_v|/2 \rceil$ and $\lfloor |P_v|/2 \rfloor$, respectively.

For convenience, we assume that no two points have the same abscissa or ordinate.

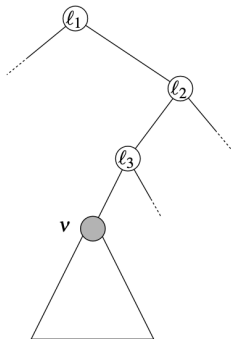
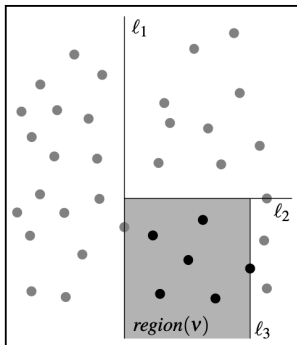


Vertical split

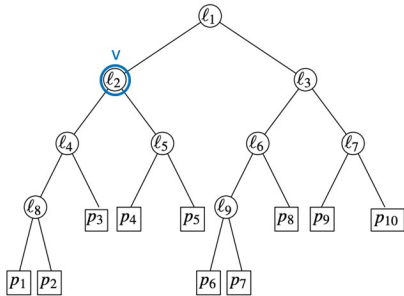
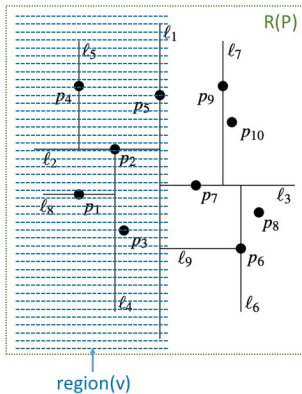


Horizontal split

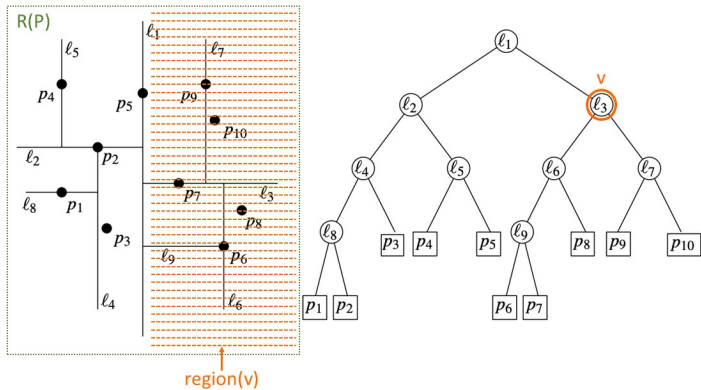
A consequence of the above definition is that for any $v \in T$, $\text{region}(v)$ is defined by the lines stored at v 's ancestors.



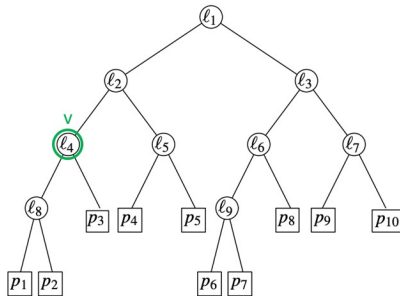
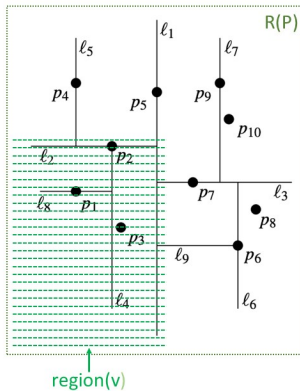
kd-tree: example



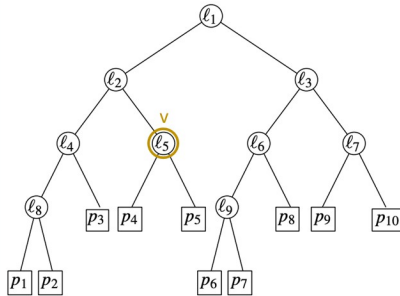
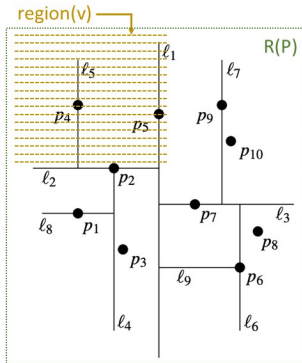
kd-tree: example



kd-tree: example



kd-tree: example



Storage requirements and construction

Let T be a kd-tree for a set $P \subset \mathbb{R}^2$ of n points.

STORAGE REQUIREMENTS. A kd-tree T for P , with $|P| = n$, can be stored using $O(n)$ space. Specifically, T contains $\leq 2n$ nodes, and

- Each internal node v stores a representation of $\text{region}(v)$ (4 coordinates suffice) and the line ℓ_v .
- Each external node v stores the point p_v .

CONSTRUCTION. Given P , T can be constructed in $O(n \log n)$ time (we skip the details).

Query

RANGE QUERY. Given a kd-tree T and a query rectangle $R = [x_1, x_2] \times [y_1, y_2]$, the **query algorithm** starts from the root and visits recursively all nodes whose regions intersect R :

- **At a leaf node v :** return p_v if $p_v \in R$.
- **At an internal node v :** recurse on each child whose associated region intersects R . If $\text{region}(v) \cap R = \emptyset$, hence none of the children's regions intersects R , return $\emptyset$.

Note that *recursion might proceed on both subtrees*.

kd-tree: query algorithm (pseudocode)

SearchKdTree(v, R) // *first invocation: $v = \text{root}$*

Input: A node v in the kd-tree, and a rectangle R

Output: The set of points $p \in R$ from the subtree of v

if (v is a leaf) **then**

if ($p_v \in R$) **then** $\text{out} \leftarrow \{p_v\};$

else $\text{out} \leftarrow \emptyset;$

else

$\text{out} \leftarrow \emptyset;$

 Let u, w be the left and right child of v , respectively;

 Let $R_u = \text{region}(u) \cap R$ and $R_w = \text{region}(w) \cap R$;

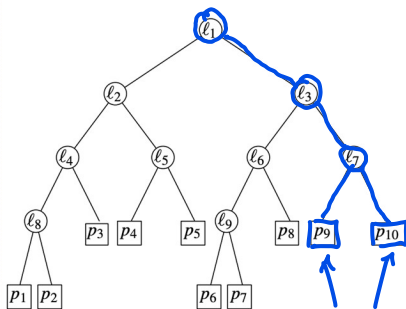
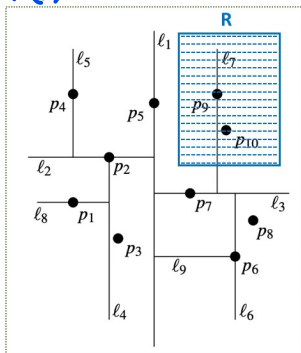
if ($R_u \neq \emptyset$) **then** $\text{out} \leftarrow \text{out} \cup \text{SearchkdTree}(u, R_u);$

if ($R_w \neq \emptyset$) **then** $\text{out} \leftarrow \text{out} \cup \text{SearchkdTree}(w, R_w);$

return out

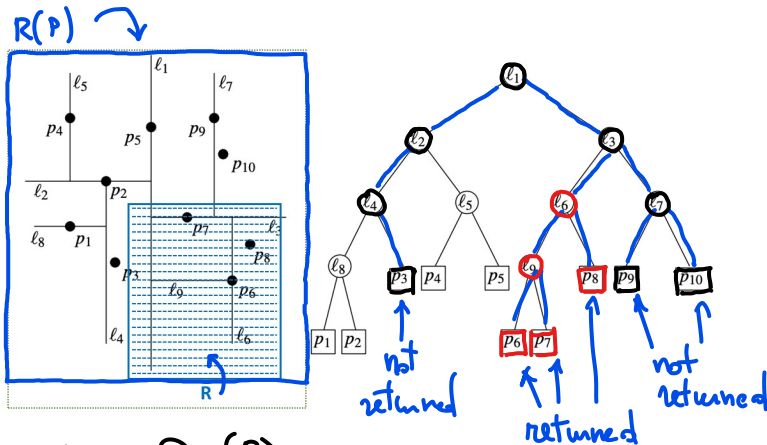
kd-tree: example of query

$R(p)$



both p_9 and p_{10}
are returned

kd-tree: example of query



Black nodes $\in Q_1(R)$

Red nodes $\in Q_2(R)$

kd-tree: performance in $\mathbb{R}^2$

Theorem

The **Range Reporting** problem for a set P of n points in $\mathbb{R}^2$ can be solved using a kd-tree T for P with the following performance

- **Construction time:** $O(n \log n)$
- **Space:** $O(n)$
- **Query time:** $O(\sqrt{n} + k)$, where k is the number of reported points.

Proof

- * Construction time space: as claimed before
- * Query time: it is proportional to the number of nodes of T visited by `SearchKdTree`. We define:

$$Q_1(R) = \{v \in T : \text{region}(v) \cap R \neq \emptyset \text{ and } \text{region}(v) \not\subseteq R\}$$

$$Q_2(R) = \{v \in T : \text{region}(v) \cap R \neq \emptyset \text{ and } \text{region}(v) \subseteq R\}$$

Important observation: `SearchKdTree` is called only on nodes in $Q_1(R) \cup Q_2(R)$ except, possibly for the first call on the root of T

Now, each cell of SearchKdTree executes $O(1)$ operations (besides the recursive calls) therefore the running time of SearchKdTree (i.e., the Query time) is $O(1 + |Q_1(R)| + |Q_2(R)|)$

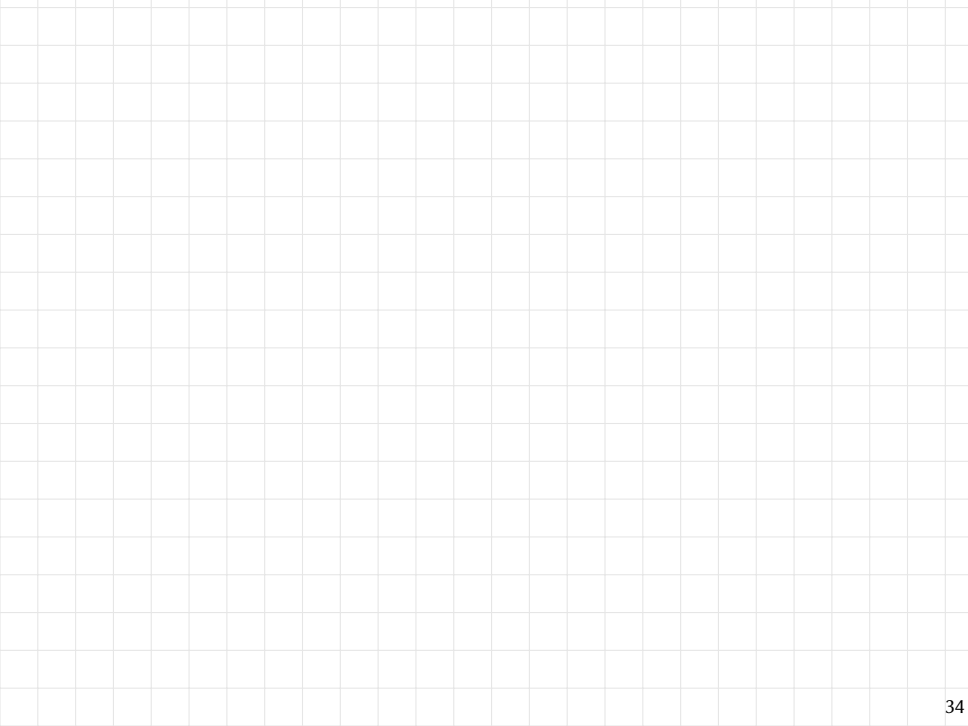
Finally, it can be shown that

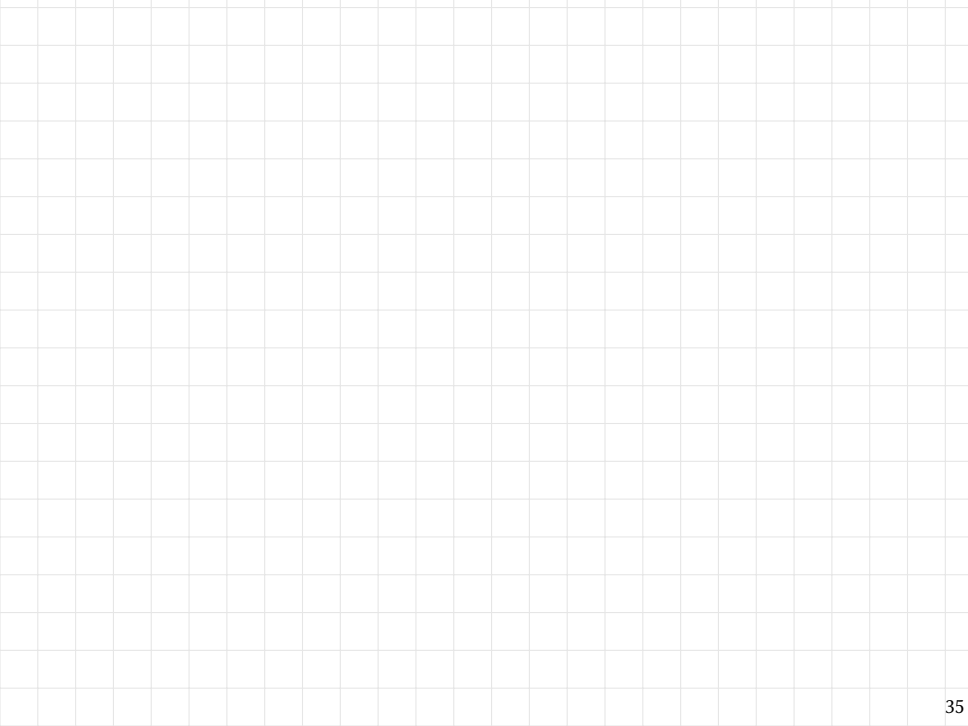
$$* |Q_1(R)| = O(\sqrt{n})$$

$$* |Q_2(R)| = O(k)$$

$\Rightarrow$ the running time of SearchKdTree is $O(\sqrt{n} + k)$

□

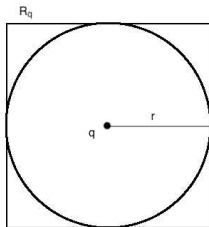




kd-tree for r -NNS

Let T be a kd-tree for a pointset $P \subset \mathbb{R}^2$. We can answer an r -NNS query with center $q \in \mathbb{R}^2$ as follows:

- Let $R_q =$ smallest square enclosing ball $B_r(q)$.



- Compute $S = \text{SearchKdTree}(T.\text{root}(), R_q)$.
- Return a point $p \in S \cap B_r(q)$, if any, or null if $S \cap B_r(q) = \emptyset$.

kd-tree for r -NNS

- The solution works efficiently from a practical point of view.
- Let $k_s = |S|$, and let $k_q = |S \cap B_r(q)|$:
 - The above solution requires $O(\sqrt{n} + k_s)$ not $O(\sqrt{n} + k_q)$
 - In some pathological cases, $k_s \gg k_q$
 - It is unclear how to (theoretically) close the gap.

Exercise

Exercise

Let P be a set of n points in $\mathbb{R}^2$ and let T be a kd-tree for P . Suppose that in each node v of T an arbitrary point p associated with one of the leaves of v 's subtree is stored. Show how to adapt SearchKdTree so that, given a query rectangle R , returns an arbitrary point $p \in P \cap R$, if one exists, or null otherwise, in time $O(\sqrt{n})$.

kd-tree: performance in $\mathbb{R}^D$

The above approach can be generalized to D dimensions.

- The kd-tree is defined in a similar fashion, but at each level the binary splits are defined by a distinct coordinate, in cycling order (i.e., $1, 2, \dots, D, 1, 2, \dots$).
- SearchKdTree remains unchanged

The following theorem generalizes the one for $\mathbb{R}^2$.

Theorem

The **Range Reporting** problem for a set P of n points in $\mathbb{R}^D$ can be solved using a kd-tree T for P with the following performance

- **Construction time:** $O(Dn \log n)$
- **Space:** $O(Dn)$
- **Query time:** $O(Dn^{1-1/D} + k)$, where k is the number of reported points.

Curse of dimensionality

- The query cost of kd-tree converges to linear scanning time when D increases.
- There is a strong conjecture that the exact r -NNS problem cannot be solved in (truly) sublinear query time when D is large, i.e., a query time $O(n^\epsilon)$ for a constant $\epsilon < 1$.